



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE INGENIERÍA ELÉCTRICA

**ANÁLISIS DE LA PERFORMANCE Y HARDWARE DE LOS MODELOS
LLMS DE GPT Y OPEN SOURCE APLICADOS A DIALOG
SUMMARIZATION**

MEMORIA PARA OPTAR AL TÍTULO DE INGENIERO CIVIL ELÉCTRICO

PABLO IGNACIO YÁÑEZ MELO

PROFESOR GUÍA:
Alberto Castro Rojas

MIEMBROS DE LA COMISIÓN:
Andrés Caba Rutte
Gonzalo Sandoval Palma

SANTIAGO DE CHILE
2025

**RESUMEN DE LA MEMORIA PARA OPTAR
AL TÍTULO DE INGENIERO CIVIL ELÉCTRICO
POR: PABLO IGNACIO YÁÑEZ MELO
FECHA: 2025
PROF. GUÍA: ALBERTO CASTRO ROJAS**

**ANÁLISIS DE LA PERFORMANCE Y HARDWARE DE LOS MODELOS
LLMS DE GPT Y OPEN SOURCE APLICADOS A DIALOG
SUMMARIZATION**

Este proyecto de grado se centra en analizar la performance y los requisitos de hardware para implementar Large Language Models (LLMs) como GPT y Open Source en la tarea de Dialog Summarization. El foco específico de la investigación es en el conjunto de datos MediaSum, que contiene extensas entrevistas televisivas y de radio en inglés. El objetivo es determinar cómo estos modelos pueden ser adaptados con recursos limitados de hardware para procesar y condensar efectivamente estos diálogos largos y complejos.

El objetivo general es analizar y evaluar la eficacia y eficiencia de modelos de lenguaje de gran escala (LLMs), incluyendo GPT-4o y alternativas de código abierto, en la tarea de resumen de diálogos, utilizando el conjunto de datos MediaSum. Los objetivos específicos incluyen desarrollar y validar métricas de evaluación para medir la precisión y coherencia de los resúmenes, identificar requisitos de hardware óptimos, realizar ajustes específicos a los modelos, comparar resultados y establecer criterios de selección basados en rendimiento y costos.

El preprocesamiento del conjunto de datos MediaSum incluyó la carga del conjunto de datos, análisis de la estructura, extracción y concatenación del historial de diálogo, conteo de tokens y cálculo de estadísticas descriptivas. El ajuste fino del modelo LLAMA2-7B-32K-Instruct se realizó utilizando técnicas de ajuste eficiente de parámetros (LoRA) y cuantización. La inferencia del modelo se llevó a cabo utilizando los conjuntos de datos de validación y se evaluó la calidad de los resúmenes generados.

La evaluación de la calidad de los resúmenes generados se realizó utilizando BERTScore y ROUGE. LLAMA2-7B-32K-Instruct mostró una mejor precisión y F1 en BERTScore, mientras que GPT-4o tuvo un mejor recall. En términos de tiempo de inferencia, GPT-4o fue más rápido, pero LLAMA2-7B-32K-Instruct ofreció una mejor relación costo-eficiencia. El costo total de inferencia para LLAMA2-7B-32K-Instruct fue significativamente menor en comparación con GPT-4o.

Este estudio ha demostrado que LLAMA2-7B-32K-Instruct ofrece una mejor relación costo-eficiencia y una calidad superior en los resúmenes generados en comparación con GPT-4o. La implementación local de modelos cuantizados en hardware local es una alternativa económica y eficiente que promete ser una solución viable para entornos con recursos computacionales limitados. El ajuste fino ha permitido capturar mejor las características específicas del conjunto de datos MediaSum, resultando en resúmenes más precisos y coherentes.

Tabla de Contenido

1. Introducción	1
1.1. Identificación y Formulación del Problema	1
1.2. Objetivos	1
1.2.1. Objetivo General	1
1.2.2. Objetivos Específicos	2
2. Metodología	3
2.1. Análisis Exploratorio de Datos y Preprocesamiento	3
2.1.1. Carga del Conjunto de Datos	3
2.1.2. Análisis de la Estructura del Conjunto de Datos	3
2.1.3. Extracción y Concatenación del Historial de Diálogo	3
2.1.4. Conteo de Tokens	3
2.1.5. Cálculo de Estadísticas Descriptivas	4
2.2. Aprendizaje por Transferencia y Fine-tuning del Modelo LLAMA2-7B-32K-Instruct	4
2.2.1. Preparación del Entorno de Trabajo	4
2.2.2. Preparación del Conjunto de Datos	4
2.2.3. Modificación del Archivo <code>dataset_info.json</code>	4
2.2.4. Entrenamiento del Modelo	4
2.2.5. Monitoreo del Proceso de Entrenamiento	4
2.3. Generación de Lenguaje Natural y Evaluación Automática de Resúmenes	4
2.3.1. Inferencia del Modelo LLAMA2	5
2.3.1.1. Carga del Modelo Ajustado	5
2.3.1.2. Preparación del Conjunto de Datos de Validación	5
2.3.1.3. Generación de Resúmenes	5
2.3.1.4. Evaluación de la Calidad de los Resúmenes	5
2.3.1.5. Análisis y Reporte de Resultados	5
2.3.2. Inferencia con el Modelo GPT-4	5
2.3.2.1. Carga del Conjunto de Datos MediaSum	5
2.3.2.2. Preparación de los Diálogos de Entrada	5
2.3.2.3. Generación de Resúmenes	5
2.3.2.4. Evaluación de la Calidad de los Resúmenes	6
2.3.2.5. Análisis y Reporte de Resultados	6

2.4.	Benchmarking y Comparación entre los Modelos	6
2.4.1.	Evaluación de Resúmenes Generados	6
2.4.2.	Análisis de Estadísticas Descriptivas	6
2.4.3.	Análisis de Eficiencia	6
2.4.3.1.	Medición del Tiempo de Inferencia	6
2.4.3.2.	Análisis Comparativo de Eficiencia	6
2.4.3.3.	Evaluación del Costo de Inferencia	7
2.5.	Elaboración de Informes y Reporte de Resultados	7
2.5.1.	Organización de los Resultados	7
2.5.2.	Presentación de Métricas de Evaluación	7
2.5.3.	Presentación de Resultados de Eficiencia	7
2.5.4.	Análisis Comparativo entre Modelos	7
2.5.5.	Discusión y Recomendaciones	7
3.	Estado del Arte	8
3.1.	Caracterización del conjunto de datos MediaSum	8
3.1.1.	Descripción del Conjunto de Datos	8
3.1.2.	Desafíos en el Resumen de Diálogos	8
3.1.3.	Implicaciones para Modelos de Resumen y Transferencia de Aprendizaje . .	9
3.2.	Definición de Resúmenes Automáticos	9
3.2.1.	Resumen Abstractivo	9
3.2.2.	Formulación Matemática de Resúmenes Abstractivos Utilizando Sequence To Sequence Conditional Generation mediante Modelos Decoder	10
3.2.2.1.	Generación de Resúmenes con un Modelo Decoder	10
3.2.2.2.	Entrenamiento de Modelos Decoder para Resumen Automático . .	11
3.3.	Caracterización del modelo LLAMA2-7B-32K-Instruct	11
3.3.1.	Estructura Transformer: Fundamentos y Relevancia	11
3.3.2.	Transformers y Self-attention	12
3.3.3.	Funcionalidad de Self-Attention y Multi-Head Attention	12
3.3.4.	Mecanismos de Atención en el Transformer	13
3.3.4.1.	Scaled Dot Product Attention	13
3.3.4.2.	Multi-Head Attention	13
3.3.5.	Arquitectura del Modelo Decoder	13
3.3.5.1.	Cálculo de la Probabilidad Condicional	13
3.3.5.2.	Estructura Base del Decoder en LLAMA2-7B-32K-Instruct	14
3.3.5.2.1	Embeddings de Tokens	15
3.3.5.2.2	Capas de Decoder	15
3.3.5.2.3	LlamaAttention	15
3.3.5.2.4	LlamaMLP	15
3.3.5.2.5	Capas de Atención y FF en Paralelo:	15
3.4.	Ajuste fino eficiente de parámetros (PEFT)	17
3.4.1.	LoRA	17

3.4.2.	Terminología y Convenciones	17
3.4.2.1.	Aplicación de LoRA en la Maximización de Probabilidades Con- dicionales	18
3.4.3.	Aplicación Práctica de LoRA en Modelos de Lenguaje	18
3.4.3.1.	Beneficios y Limitaciones Prácticos	18
3.4.4.	QLoRA	19
3.4.4.1.	Cuatificación en Bloque de k-bits	19
3.4.4.2.	Adaptadores de Bajo Rango	20
3.4.4.3.	Requisitos de Memoria y Optimizadores Paginados	20
3.4.4.4.	Definición Formal de QLoRA	20
3.5.	Métricas de Evaluación	21
3.5.1.	ROUGUE	22
3.5.2.	BERTScore	22
3.5.2.1.	Representación de Tokens y Cálculo de Similitud	22
3.5.2.2.	Cálculo de BERTScore	23
4.	Resultados	24
4.1.	Descripción General	24
4.2.	Ingeniería de prompts	24
4.2.1.	LLAMA2-7B-32K-Instruct	24
4.2.2.	GPT-4o	26
4.3.	Descripción del conjunto de Validación	27
4.3.1.	Tokens en el conjunto de Validación	27
4.3.2.	Discusión acerca de los tokens	28
4.4.	Resultados del Fine-Tuning	28
4.4.1.	Eficiencia del Hardware y Throughput de Tokens	29
4.5.	Evaluación de la Calidad de los Resúmenes	29
4.5.1.	Resultados de BERTScore	29
4.5.2.	Resultados de ROUGE	30
4.6.	Evaluación de la Eficiencia de los Modelos	30
4.6.1.	Tiempo de Inferencia	30
4.6.2.	Costo de Inferencia	30
4.7.	Análisis Comparativo entre Modelos	31
5.	Discusión y Trabajo Futuro	32
5.1.	Contribuciones	34
5.1.1.	Impacto Económico y Práctico	34
5.1.2.	Avances en el Conocimiento y Habilidades Técnicas	34
6.	Conclusión	35
	Bibliografía	37

Índice de Tablas

3.1.	Caracterización mediante estadísticos del conjunto de datos MediaSum. [1]	8
4.1.	Comparación de tokens en el conjunto de validación para GPT-4o y LLAMA2-7b-32K-Instruct	28
4.2.	Resultados de la pérdida y precisión durante el entrenamiento y evaluación	28
4.3.	Resultados de BERTScore para LLAMA y GPT	29
4.4.	Resultados de ROUGE para LLAMA y GPT	30
4.5.	Tiempo de Inferencia para LLAMA y GPT	30
4.6.	Costo de Inferencia para GPT	30
4.7.	Costo de Inferencia para LLAMA	31

Índice de Ilustraciones

3.1.	Vista general de los modelos de resumen abstractivo basados en Transformer [2]	10
3.2.	Arquitectura Transformer [2]	12
3.3.	Scale Dot-Product Attention y Multi-Head Attention [2]	14
3.4.	Esquema de LoRA durante y post entrenamiento	19
3.5.	Esquema de Ajuste Estandar v/s LoRA v/s QLoRA	19
3.6.	Computación de BERTScore [2]	23
4.1.	Pérdida de entrenamiento del modelo LLAMA2-chat-7b	29

Capítulo 1

Introducción

1.1. Identificación y Formulación del Problema

Este proyecto de grado se centra en analizar la performance y los requisitos de hardware para implementar Large Language Models (LLMs) como GPT y Open Source en la tarea de Dialog Summarization. El foco específico de la investigación es en el conjunto de datos MediaSum, que contiene extensas entrevistas televisivas y de radio en inglés. El objetivo es determinar cómo estos modelos pueden ser adaptados con recursos limitados de hardware para procesar y condensar efectivamente estos diálogos largos y complejos. La relevancia de este estudio radica en su potencial para transformar la forma en que se procesan y se comprenden grandes volúmenes de información verbal, facilitando un acceso más rápido, menos costoso y más preciso a las ideas principales de conversaciones.

La metodología propuesta incluye la implementación y evaluación comparativa de modelos LLM, en particular LLAMA2-7b-32K-Instruct, centrándose en aspectos clave como la precisión del resumen, la capacidad de capturar el contenido esencial de las conversaciones, y la eficiencia en el uso de recursos de hardware. Esta investigación no solo contempla la viabilidad técnica y la eficiencia de estos sistemas, sino también considera los costos asociados y busca alternativas más accesibles. La memoria concluirá evaluando la aplicabilidad de los modelos LLM para resumir diálogos, ofreciendo una perspectiva detallada sobre su potencial y limitaciones.

1.2. Objetivos

1.2.1. Objetivo General

Analizar y evaluar la eficacia y eficiencia de modelos de lenguaje de gran escala (LLMs), incluyendo GPT-4o y alternativas una alternativa de código abierto, en la tarea de resumen de diálogos, utilizando el conjunto de datos MediaSum. Desarrollar los modelos LLAMA2-7b-32K-Instruct con técnicas de ajuste eficiente de parámetros para el resumen de diálogos y sus métricas de evaluación de performance.

1.2.2. Objetivos Específicos

1. Desarrollar y validar métricas de evaluación para medir la precisión y coherencia de los resúmenes generados por los modelos LLM.
2. Identificar y caracterizar los requisitos de hardware óptimos para la implementación eficiente de estos modelos.
3. Realizar ajustes específicos (fine-tuning) a los modelos LLM para adaptarlos a las características de los diálogos extensos en MediaSum.
4. Comparar los resultados de los modelos LLM ajustados con GPT-4o, enfocándose en la calidad del resumen y el rendimiento del hardware.
5. Establecer criterios de selección para determinar el modelo más adecuado, basándose en un equilibrio entre rendimiento y costos.

Capítulo 2

Metodología

2.1. Análisis Exploratorio de Datos y Preprocesamiento

2.1.1. Carga del Conjunto de Datos

El primer paso en el preprocesamiento es cargar el conjunto de datos MediaSum desde la biblioteca `datasets`. Este conjunto de datos contiene diálogos extensos de entrevistas televisivas y de radio, los cuales deben ser preprocesados para ser utilizados en la tarea de resumen de diálogos.

2.1.2. Análisis de la Estructura del Conjunto de Datos

Cada entrada en el conjunto de datos MediaSum contiene información sobre el diálogo original y un resumen correspondiente. Es crucial extraer y estructurar esta información de manera que pueda ser utilizada eficientemente por los modelos de lenguaje.

2.1.3. Extracción y Concatenación del Historial de Diálogo

Para cada entrada en el conjunto de datos, se extrae el historial de diálogo y el resumen correspondiente. El historial de diálogo se compone de múltiples secuencias de texto que representan las intervenciones de los participantes en la entrevista. Estas secuencias son concatenadas para formar un solo texto de entrada para el modelo.

2.1.4. Conteo de Tokens

Una vez que se ha extraído y concatenado el historial de diálogo, se cuenta el número de tokens en el texto de entrada y en el resumen. Esto permite evaluar la longitud de los textos y ajustar los parámetros del modelo en consecuencia. La biblioteca `tiktoken` se utiliza para realizar esta tarea.

2.1.5. Cálculo de Estadísticas Descriptivas

Finalmente, se calculan las estadísticas descriptivas de los tokens, como la suma total, el máximo y el promedio de tokens tanto para los textos de entrada como para los resúmenes. Estas estadísticas proporcionan una visión general de la complejidad del conjunto de datos y ayudan a optimizar el proceso de entrenamiento y evaluación de los modelos.

2.2. Aprendizaje por Transferencia y Fine-tuning del Modelo LLAMA2-7B-32K-Instruct

2.2.1. Preparación del Entorno de Trabajo

El primer paso en el fine-tuning es preparar el entorno de trabajo. Se clona el repositorio de LLaMA-Factory y se instalan las dependencias necesarias para el entrenamiento del modelo.

2.2.2. Preparación del Conjunto de Datos

Una vez que el entorno está preparado, se procede con la preparación del conjunto de datos para el fine-tuning. Esto implica generar un archivo JSON que contenga las instrucciones y los diálogos concatenados del conjunto de datos MediaSum, tanto para el conjunto de entrenamiento como para el de validación.

2.2.3. Modificación del Archivo `dataset_info.json`

El siguiente paso es modificar el archivo `dataset_info.json` para incluir la información sobre los archivos JSON generados. Este archivo debe especificar las columnas correspondientes a las instrucciones y las respuestas en el conjunto de datos.

2.2.4. Entrenamiento del Modelo

Con el conjunto de datos preparado, se procede con el entrenamiento del modelo utilizando la herramienta `llamafactory-cli`. Durante el entrenamiento, se emplean técnicas de ajuste eficiente de parámetros (LoRA) y cuantización, lo que permite optimizar el uso de recursos de hardware.

2.2.5. Monitoreo del Proceso de Entrenamiento

Finalmente, se monitorea el proceso de entrenamiento para asegurarse de que el modelo se está ajustando correctamente. Se registran las métricas de rendimiento y se guardan los *checkpoints* del modelo para su posterior evaluación.

2.3. Generación de Lenguaje Natural y Evaluación Automática de Resúmenes

2.3.1. Inferencia del Modelo LLAMA2

2.3.1.1. Carga del Modelo Ajustado

El primer paso en la inferencia es cargar el modelo LLAMA2 previamente entrenado y ajustado. Es crucial asegurarse de que todos los parámetros y configuraciones del modelo estén correctamente cargados para garantizar resultados precisos.

2.3.1.2. Preparación del Conjunto de Datos de Validación

El siguiente paso es preparar el conjunto de datos de validación para la inferencia. Se extraen los diálogos y resúmenes del conjunto de datos MediaSum y se estructuran de manera que puedan ser procesados por el modelo LLAMA2.

2.3.1.3. Generación de Resúmenes

Durante el proceso de inferencia, se utiliza el modelo LLAMA2 para generar resúmenes a partir de los diálogos de entrada. Es importante monitorear el rendimiento del modelo y ajustar los parámetros de inferencia según sea necesario para optimizar la calidad de los resúmenes generados.

2.3.1.4. Evaluación de la Calidad de los Resúmenes

Una vez generados los resúmenes, se procede a evaluar su calidad utilizando métricas como ROUGE y BERTScore. Estas métricas permiten comparar los resúmenes generados con los resúmenes reales y cuantificar la precisión y coherencia de los resultados.

2.3.1.5. Análisis y Reporte de Resultados

Finalmente, se analizan los resultados de la inferencia y se generan reportes detallados que incluyen las métricas de rendimiento y las estadísticas descriptivas de los resúmenes generados. Estos reportes son fundamentales para evaluar la efectividad del modelo LLAMA2 en la tarea de resumen de diálogos.

2.3.2. Inferencia con el Modelo GPT-4

2.3.2.1. Carga del Conjunto de Datos MediaSum

El primer paso en la inferencia es cargar el conjunto de datos MediaSum. Este conjunto de datos contiene diálogos extensos que deben ser procesados para generar resúmenes precisos y coherentes.

2.3.2.2. Preparación de los Diálogos de Entrada

El siguiente paso es preparar los diálogos de entrada para el modelo GPT-4. Esto implica concatenar las secuencias de diálogo y contar el número de tokens en el texto de entrada. Este conteo es crucial para ajustar los parámetros del modelo y optimizar el proceso de inferencia.

2.3.2.3. Generación de Resúmenes

Durante la inferencia, se utiliza el modelo GPT-4 para generar resúmenes a partir de los diálogos de entrada. Es importante monitorear el rendimiento del modelo y ajustar los parámetros de

inferencia según sea necesario para obtener resúmenes de alta calidad.

2.3.2.4. Evaluación de la Calidad de los Resúmenes

Una vez generados los resúmenes, se procede a evaluar su calidad utilizando métricas como ROUGE y BERTScore. Estas métricas permiten comparar los resúmenes generados con los resúmenes reales y cuantificar su precisión y coherencia.

2.3.2.5. Análisis y Reporte de Resultados

Finalmente, se analizan los resultados de la inferencia y se generan reportes detallados que incluyen las métricas de rendimiento y las estadísticas descriptivas de los resúmenes generados. Estos reportes son fundamentales para evaluar la efectividad del modelo GPT-4 en la tarea de resumen de diálogos.

2.4. Benchmarking y Comparación entre los Modelos

2.4.1. Evaluación de Resúmenes Generados

La comparación entre los modelos GPT-4 y LLAMA2-7B-32K es esencial para evaluar su efectividad y eficiencia en la tarea de resumen de diálogos. Se utilizan métricas como ROUGE y BERTScore para comparar la calidad de los resúmenes generados. Estas métricas permiten cuantificar la precisión, coherencia y relevancia de los resúmenes en relación con los resúmenes reales.

2.4.2. Análisis de Estadísticas Descriptivas

Además de las métricas de evaluación, se analizan las estadísticas descriptivas de los tokens generados por ambos modelos. Esto incluye la suma total, el máximo y el promedio de tokens en los textos de entrada y los resúmenes generados.

2.4.3. Análisis de Eficiencia

2.4.3.1. Medición del Tiempo de Inferencia

El primer paso es medir el tiempo de inferencia para cada modelo. Esto incluye el tiempo total necesario para generar los resúmenes a partir de los diálogos de entrada. La medición del tiempo de inferencia es fundamental para evaluar la rapidez con la que los modelos pueden procesar grandes volúmenes de datos.

2.4.3.2. Análisis Comparativo de Eficiencia

Se realiza un análisis comparativo del tiempo de inferencia y el uso de recursos de hardware entre los modelos GPT-4 y LLAMA2-7B-32K-Instruct. Este análisis permite identificar las fortalezas y debilidades de cada modelo en términos de eficiencia.

2.4.3.3. Evaluación del Costo de Inferencia

Además de las métricas de tiempo y uso de recursos, es importante evaluar el costo asociado con la inferencia de cada modelo. Esto incluye el costo de procesamiento en términos de consumo de energía y el costo financiero de utilizar recursos de hardware específicos.

2.5. Elaboración de Informes y Reporte de Resultados

2.5.1. Organización de los Resultados

El primer paso es organizar los resultados obtenidos de las diferentes evaluaciones en un formato estructurado. Esto incluye las métricas de evaluación de la calidad de los resúmenes, las estadísticas descriptivas de los tokens y los resultados del análisis de eficiencia.

2.5.2. Presentación de Métricas de Evaluación

Las métricas de evaluación de la calidad de los resúmenes son ROUGE y BERTScore, las cuales son calculadas individualmente y se genera un resumen de los promedios obtenidos.

2.5.3. Presentación de Resultados de Eficiencia

Además de las métricas de calidad, el reporte incluye los resultados del análisis de eficiencia. Esto incluye el tiempo de inferencia, el uso de recursos de hardware y el costo asociado con la inferencia de cada modelo. Estos resultados son cruciales para evaluar la viabilidad de implementar los modelos en diferentes entornos.

2.5.4. Análisis Comparativo entre Modelos

El reporte es un análisis comparativo entre los modelos GPT-4 y LLAMA2-7B-32K-Instruct. Este análisis resalta las fortalezas y debilidades de cada modelo en términos de calidad y eficiencia, proporcionando una visión equilibrada de su rendimiento.

2.5.5. Discusión y Recomendaciones

Finalmente, se concluye con una discusión de los resultados y recomendaciones para futuras investigaciones. Esta discusión aborda las limitaciones del estudio, las posibles mejoras en la metodología y las implicaciones prácticas de los resultados obtenidos.

Capítulo 3

Estado del Arte

3.1. Caracterización del conjunto de datos MediaSum

MediaSum es un conjunto de datos de gran escala destinado a la tarea de resumen de diálogos en medios de comunicación. Consiste en 463.6K transcripciones de entrevistas con resúmenes abstractivos, recolectadas de fuentes como NPR y CNN. Este conjunto de datos representa un avance significativo en el campo del resumen de diálogos debido a su magnitud y la complejidad de las conversaciones que incluye.

3.1.1. Descripción del Conjunto de Datos

MediaSum es notable por su tamaño, superando en una orden de magnitud a los conjuntos de datos públicos previos en el dominio del resumen de diálogos. Incluye transcripciones de entrevistas de radio y televisión, cubriendo una amplia gama de temas. Cada transcripción en NPR viene acompañada de un resumen general, mientras que las transcripciones de CNN están segmentadas por temas con descripciones que sirven como resúmenes. [1]

Tabla 3.1: Caracterización mediante estadísticos del conjunto de datos MediaSum. [1]

Estadísticas	NPR	CNN
Diálogos	49,420	414,176
Promedio de palabras por diálogo	906.3	1,630.9
Promedio de palabras por resumen	40.2	11.3
Turnos	24.2	30.7
Hablantes	4.0	6.8
Palabras nuevas en resumen	33.6 %	24.9 %

3.1.2. Desafíos en el Resumen de Diálogos

El resumen de diálogos en MediaSum presenta varios desafíos únicos. Por ejemplo, las conversaciones multi-parte y la diversidad temática requieren un enfoque sofisticado para identificar y

resumir eficazmente los puntos clave. Además, en [1] se observa que hay un sesgo posicional en los diálogos, con palabras clave que suelen mencionarse al principio y al final de las entrevistas de radio, mientras que en las televisadas suelen aparecer al principio.

3.1.3. Implicaciones para Modelos de Resumen y Transferencia de Aprendizaje

Los experimentos realizados en [1] con MediaSum demuestran su utilidad para mejorar el rendimiento de los modelos de resumen en tareas de resumen de diálogos. La amplia variedad de conversaciones y la estructura única de los resúmenes proporcionan un terreno fértil para el entrenamiento y la afinación de modelos avanzados de resumen automático, así como para la transferencia de aprendizaje a otros conjuntos de datos.

3.2. Definición de Resúmenes Automáticos

El resumen automático ha ganado una importancia significativa en el campo del procesamiento del lenguaje natural, ofreciendo soluciones prácticas para condensar textos extensos en versiones más cortas y manejables. Mientras que existen diversas técnicas como la extractiva, la abstractiva y la híbrida, este documento se centrará en la técnica abstractiva. Esta técnica es fundamental en aplicaciones avanzadas de inteligencia artificial y procesamiento de lenguaje natural, destacando por su capacidad para generar resúmenes que reflejan el contenido esencial del texto original, pero con formulaciones y sintaxis únicas.

3.2.1. Resumen Abstractivo

El resumen abstractivo utiliza algoritmos de inteligencia artificial para crear un nuevo texto que, aunque conserva el contenido esencial y el significado del original, no replica necesariamente las mismas frases o estructuras sintácticas. Esta capacidad de reformulación y síntesis lo distingue claramente de los métodos extractivos y híbridos. Un ejemplo destacado en la investigación actual es el modelo LLAMA2, que ilustra la eficacia y las capacidades avanzadas de la técnica abstractiva en la generación de resúmenes. [2]

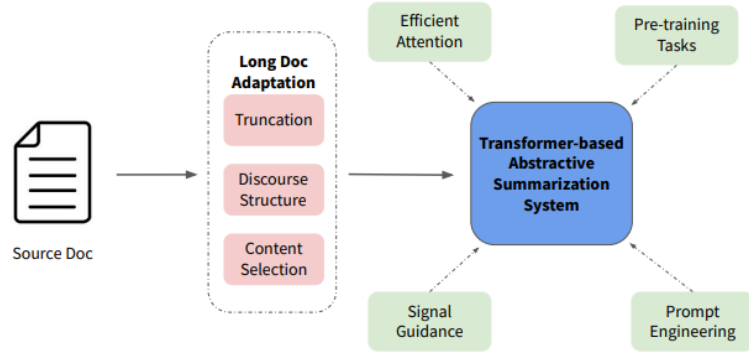


Figura 3.1: Vista general de los modelos de resumen abstractivo basados en Transformer [2]

3.2.2. Formulación Matemática de Resúmenes Abstractivos Utilizando Sequence To Sequence Conditional Generation mediante Modelos Decoder

La tarea de generar resúmenes automáticos puede ser formalizada matemáticamente utilizando un modelo decoder, como se encuentra en arquitecturas de modelos de lenguaje tales como GPT-4o y LLAMA2-7B-32K-Instruct. Esta formulación se centra en la generación de una secuencia de tokens que represente de manera eficaz y concisa el contenido de un documento fuente.

3.2.2.1. Generación de Resúmenes con un Modelo Decoder

La definición formal de los resúmenes automáticos en el contexto de la Dialog Summarization puede ser modelada matemáticamente considerando tanto el documento original como el resumen de referencia. Para un documento D y un resumen correspondiente S , cada documento tiene una secuencia de tokens $D_{\text{token}} = \{d_1, d_2, \dots, d_m\}$ y cada resumen tiene también una secuencia de tokens $S_{\text{token}} = \{s_1, s_2, \dots, s_n\}$.

El objetivo es generar una secuencia de tokens $S = \langle s_1^*, \dots, s_i^*, \dots, s_n^* \rangle$ que resuma la información contenida en un documento fuente D . Aquí, cada token s_i es seleccionado de un vocabulario de palabras $\mathcal{V}$. Formalmente, la tarea del decodificador se describe como:

$$S \leftarrow g(D)$$

Donde se denota la distribución de probabilidad p_g parametrizada por el decoder g como:

$$p_g(S \mid D) = p(s_1, \dots, s_i, \dots, s_n \mid D)$$

Donde g representa al modelo auto-regresivo o decodificador que elige cada token de la secuencia de resumen, condicionado por el documento fuente D . La probabilidad de un resumen candidato S dado el documento fuente D se factoriza de la siguiente manera:

$$p_g(S \mid D) = \prod_{i=1}^{l_S} p_g(s_i \mid S_{<i}, D),$$

donde s_i es el i -ésimo token en S , $S_{<i}$ es la secuencia de tokens antes de s_i , y l_S es la longitud de la secuencia de resumen S .

3.2.2.2. Entrenamiento de Modelos Decoder para Resumen Automático

El algoritmo de entrenamiento estándar para estos modelos es la Estimación de Máxima Verosimilitud (MLE) utilizando un resumen de referencia S^* . Con la formulación anterior, la optimización MLE en este caso es equivalente a minimizar la siguiente pérdida de entropía cruzada:

$$\mathcal{L}(\theta) = -\log p_g(S^* \mid D; \theta) = -\sum_{i=1}^{l_{S^*}} \log p_g(s_i^* \mid S_{<i}^*, D; \theta),$$

donde θ son los parámetros aprendibles del modelo g . Esta formulación asume una distribución objetivo puntual p^* en el resumen de referencia S^* , esencial para guiar al modelo en la generación de resúmenes que se alinean con los estándares de referencia humana.

3.3. Caracterización del modelo LLAMA2-7B-32K-Instruct

3.3.1. Estructura Transformer: Fundamentos y Relevancia

La arquitectura Transformer, introducida en el trabajo seminal "Attention Is All You Need" por Illia Polosukhin y otros [3] cuyo trabajo se resume a continuación, ha revolucionado el campo de los modelos de lenguaje basados en aprendizaje profundo. Esta arquitectura se distingue por su enfoque único en mecanismos de atención, prescindiendo completamente de la recurrencia y las convoluciones que caracterizaban a los modelos anteriores.

En los modelos Transformer, la atención se utiliza para captar dependencias globales entre la entrada y la salida, permitiendo que cada posición en la secuencia de entrada contribuya directamente a cada posición en la secuencia de salida. Esto facilita un paralelismo significativamente mayor en el procesamiento, lo que reduce los tiempos de entrenamiento y mejora la eficiencia general del modelo.

Además, la arquitectura Transformer ha demostrado ser superior en calidad en tareas como la traducción automática, logrando puntajes BLEU significativamente más altos que los modelos anteriores. Su capacidad para generalizar a otras tareas, como el análisis sintáctico del inglés, también ha sido demostrada, lo que indica su versatilidad y eficacia en una amplia gama de aplicaciones de procesamiento del lenguaje natural.

La importancia de la arquitectura Transformer radica en su habilidad para manejar secuencias largas y su eficiencia en capturar relaciones contextuales complejas en el texto, lo que la hace ideal para el desarrollo de modelos de lenguaje avanzados como LLAMA2.

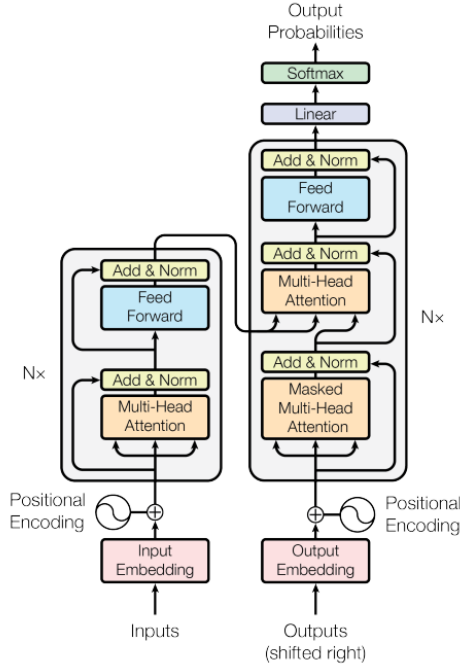


Figura 3.2: Arquitectura Transformer [2]

3.3.2. Transformers y Self-attention

Los transformers se basan en el concepto de auto-atención, también conocida como self-attention. Esta forma de atención calcula representaciones de una secuencia al relacionar diferentes posiciones de esa misma secuencia. La auto-atención ha sido utilizada con éxito en una variedad de tareas, incluyendo la comprensión de lectura, la resumen abstractivo, la inferencia textual y el aprendizaje de representaciones de oraciones independientes de la tarea.

Los modelos basados en transformadores como el LLAMA2-7B-32K-Instruct se destacan por su capacidad para computar representaciones de sus entradas y salidas utilizando únicamente la auto-atención, sin depender de redes neuronales recurrentes o convolucionales alineadas con la secuencia.

3.3.3. Funcionalidad de Self-Attention y Multi-Head Attention

El mecanismo de atención en los transformers puede describirse como un mapeo de una consulta ó "query" y un conjunto de pares clave-valor ó "key-values" a una salida, donde la salida se calcula como una suma ponderada de los valores. Estos pesos se determinan mediante una función de compatibilidad entre la consulta y las claves correspondientes.

En la práctica, la función de atención se calcula para un conjunto de queries simultáneamente, agrupadas en una matriz Q , con las keys y values también agrupados en matrices K y V , respectivamente. Este enfoque facilita una implementación eficiente y optimizada del modelo.

Además, los transformadores emplean "Multi-Head Attention", donde varias capas de atención funcionan en paralelo, permitiendo al modelo atender conjuntamente a información de diferentes subespacios representativos en diferentes posiciones. Esto enriquece la capacidad del modelo para

interpretar y procesar la secuencia de entrada.

3.3.4. Mecanismos de Atención en el Transformer

3.3.4.1. Scaled Dot Product Attention

El modelo Transformer implementa un mecanismo de atención conocido como "Scaled Dot Product Attention". La entrada de este mecanismo consiste en consultas (queries) y claves (keys) de dimensión d_k y valores (values) de dimensión d_v . Los productos escalares de la consulta con todas las claves se calculan, se dividen por $\sqrt{d_k}$ y se aplica una función softmax para obtener los pesos sobre los valores. La fórmula es:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Este mecanismo es más rápido y eficiente en términos de espacio que las versiones anteriores de atención, gracias a la posibilidad de utilizar código de multiplicación de matrices en paralelo.

3.3.4.2. Multi-Head Attention

El Transformer utiliza "Atención Multi-Cabeza" para mejorar su capacidad de atención. En lugar de realizar una única función de atención con claves, valores y consultas de dimensión d_{model} , proyecta linealmente las consultas, claves y valores h veces con diferentes proyecciones lineales aprendidas a dimensiones d_k , d_k y d_v , respectivamente. Luego, ejecuta la función de atención en cada una de estas versiones proyectadas en paralelo, produciendo valores de dimensión d_v . Las salidas se concatenan y se proyectan una vez más, resultando en los valores finales. La fórmula es:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

donde $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ y $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ son las matrices de pesos de la red.

La Atención Multi-Cabeza permite que el modelo preste atención conjunta a información de diferentes subespacios representacionales y posiciones, una capacidad clave para el procesamiento efectivo del lenguaje natural.

3.3.5. Arquitectura del Modelo Decoder

El modelo LLAMA2-7B implementa una arquitectura avanzada de decodificador, construida sobre los principios del modelo Transformer. Esta sección describe detalladamente las características y funcionalidades clave de este decodificador, proporcionando una visión completa de su estructura y capacidades en el contexto de la generación de texto y, en particular, el resumen de diálogos.

3.3.5.1. Cálculo de la Probabilidad Condicional

En el modelo LLAMA2-7B, la probabilidad condicional de un token se calcula a partir de las interacciones entre las capas de embedding y el mecanismo de Multi-Head Attention. Los embed-

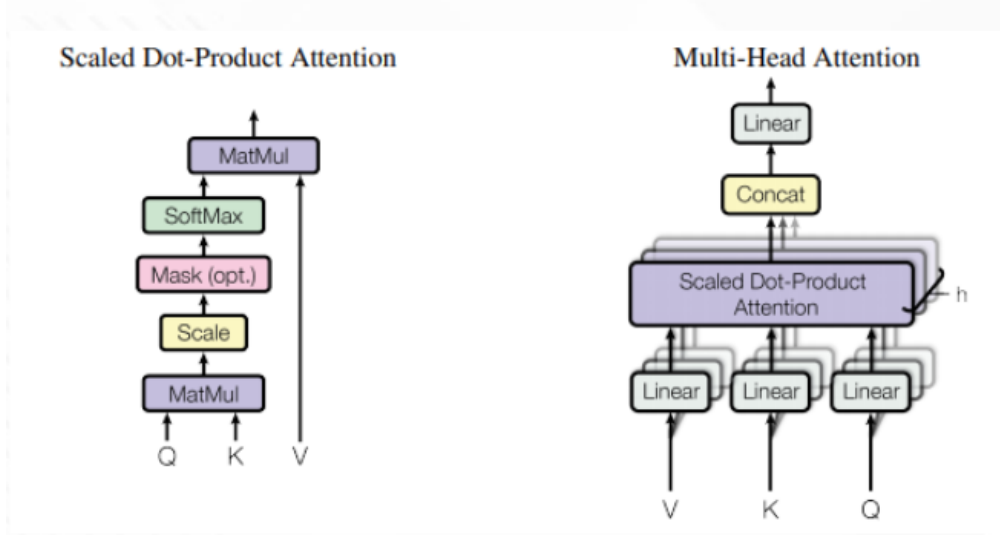


Figura 3.3: Scale Dot-Product Attention y Multi-Head Attention [2]

dings de tokens transforman cada token de entrada en representaciones vectoriales densas, que son los inputs para las capas de atención.

El mecanismo de Multi-Head Attention en el modelo procesa estas representaciones para calcular la probabilidad condicional de cada token siguiente. La fórmula general de Multi-Head Attention es:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

donde cada head_i es calculado como:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

En este contexto, Q , K , y V representan las matrices de queries, keys, y values derivadas de las representaciones de embedding. La atención se aplica a estas representaciones para calcular los pesos que determinan la importancia relativa de cada token en la secuencia.

La probabilidad condicional de un token siguiente s_i dado el contexto previo $S_{<i}$ y el documento fuente D se calcula como:

$$p_g(s_i | S_{<i}, D) = \text{softmax}(\text{Feed-Forward}(\text{MultiHead}(Q, K, V)))$$

Este cálculo integra la información contextual proporcionada por las representaciones de embedding y los pesos de atención generados por el mecanismo de Multi-Head Attention para predecir el siguiente token en la secuencia pasando por una capa fully connected y la softmax.

3.3.5.2. Estructura Base del Decoder en LLAMA2-7B-32K-Instruct

La arquitectura del decoder en LLAMA2-7B [4] se caracteriza por su enfoque en capas de atención y procesamiento de lenguaje, esenciales para la generación de texto. El modelo incorpora:

3.3.5.2.1. Embeddings de Tokens

Una capa de embedding que transforma tokens de entrada en representaciones vectoriales densas. El vocabulario es fijo siendo 32000 tokens para este modelo.

3.3.5.2.2. Capas de Decoder

Múltiples capas de decoder, cada una compuesta por mecanismos de atención y redes de perceptrón multicapa (MLP).

Cada capa de decoder en LLAMA2-7B incluye:

3.3.5.2.3. LlamaAttention

Un mecanismo de atención especializado que procesa las representaciones de entrada para capturar dependencias contextuales.

3.3.5.2.4. LlamaMLP

Una red MLP que realiza transformaciones adicionales en las salidas de la atención para refinar aún más las representaciones.

3.3.5.2.5. Capas de Atención y FF en Paralelo:

El modelo ejecuta capas de atención y feed-forward en paralelo y suma los resultados.

Código 3.1: Estructura del Modelo LLAMA2-7B.

```
1 {
2   "LlamaForCausalLM": {
3     "model": "LlamaModel",
4     "components": {
5       "embed_tokens": "Embedding(32000, 4096)",
6       "layers": {
7         "type": "ModuleList",
8         "details": [
9           "32 x LlamaDecoderLayer",
10          {
11            "self_attn": "LlamaAttention",
12            "components": {
13              "q_proj": "Linear(in_features=4096, out_features=4096, bias=False)",
14              "k_proj": "Linear(in_features=4096, out_features=4096, bias=False)",
15              "v_proj": "Linear(in_features=4096, out_features=4096, bias=False)",
16              "o_proj": "Linear(in_features=4096, out_features=4096, bias=False)",
17              "rotary_emb": "LlamaRotaryEmbedding()"
18            }
19          },
20          {
21            "mlp": "LlamaMLP",
```

```

22     "components": {
23         "gate_proj": "Linear(in_features=4096, out_features=11008, bias=False)",
24         "up_proj": "Linear(in_features=4096, out_features=11008, bias=False)",
25         "down_proj": "Linear(in_features=11008, out_features=4096, bias=False)",
26         "act_fn": "SiLUActivation()"
27     }
28 },
29 {
30     "input_layernorm": "LlamaRMSNorm()",
31     "post_attention_layernorm": "LlamaRMSNorm()"
32 }
33 ]
34 },
35 "norm": "LlamaRMSNorm()"
36 },
37 "lm_head": "Linear(in_features=4096, out_features=32000, bias=False)"
38 }
39 }

```

3.4. Ajuste fino eficiente de parámetros (PEFT)

3.4.1. LoRA

En el ámbito de la adaptación de modelos de lenguaje a gran escala, como GPT-3, para múltiples aplicaciones, se presenta un desafío crítico: el ajuste fino de estos modelos implica actualizar una cantidad masiva de parámetros, lo cual resulta en una carga significativa en términos de almacenamiento y eficiencia operacional.

Para abordar esta problemática, introducimos la metodología de Ajuste Fino Eficiente de Parámetros (PEFT) a través de la aproximación de Adaptación de Bajo Rango (LoRA) [5]. LoRA reformula el proceso de adaptación no como una actualización de todos los parámetros pre-entrenados, sino optimizando matrices de descomposición de rango de los cambios en las capas densas durante la adaptación, manteniendo congelados los pesos pre-entrenados. Esto reduce significativamente la cantidad de almacenamiento y la sobrecarga computacional.

Las ventajas clave de LoRA incluyen:

- **Compartición y Modularidad:** Un modelo pre-entrenado puede ser compartido y utilizado para construir múltiples módulos LoRA para diferentes tareas, intercambiando eficientemente las matrices A y B para cada tarea, lo cual reduce considerablemente los requisitos de almacenamiento y la sobrecarga en el cambio de tareas.
- **Eficiencia de Entrenamiento y Accesibilidad:** Al optimizar solo las matrices de bajo rango inyectadas, LoRA hace el entrenamiento más eficiente y reduce la barrera de hardware para la entrada, evitando el cálculo de gradientes o el mantenimiento de estados del optimizador para la mayoría de los parámetros.
- **Nula Latencia Adicional en la Inferencia:** El diseño lineal y sencillo de LoRA permite fusionar las matrices entrenables con los pesos congelados durante la implementación, no introduciendo latencia adicional en comparación con un modelo completamente ajustado.
- **Compatibilidad y Flexibilidad:** LoRA es compatible con muchas metodologías anteriores y puede combinarse con ellas, como la afinación de prefijos, ampliando así su aplicabilidad en diversos contextos de adaptación de modelos.

3.4.2. Terminología y Convenciones

En nuestro enfoque, hacemos referencias frecuentes a la arquitectura Transformer y utilizamos las terminologías convencionales para sus dimensiones. Denominamos la dimensión de entrada y salida de una capa Transformer como d_{model} . Usamos W^Q, W^K, W^V , y W^O para referirnos a las matrices de proyección de consulta/clave/valor/salida en el módulo de auto-atención. W o W^O se refiere a una matriz de peso pre-entrenado y ΔW a su actualización de gradiente acumulada durante la adaptación. Utilizamos r para denotar el rango de un módulo LoRA.

3.4.2.1. Aplicación de LoRA en la Maximización de Probabilidades Condicionales

Considerando el ajuste fino eficiente de parámetros con LoRA en el contexto de la generación de resúmenes automáticos, ajustamos las fórmulas para reflejar de manera coherente la notación utilizada en el documento. La fórmula original para la optimización de parámetros en el modelado de lenguaje, considerando el conjunto de datos (D, S^*) como $\mathcal{Z}$ y el modelo decoder g parametrizado por θ es:

$$\max_{\theta} \sum_{(D, S^*) \in \mathcal{Z}} \sum_{i=1}^{l_{S^*}} \log(p_{\theta}(s_i^* | S_{<i}^*, D))$$

Bajo el enfoque de LoRA, esta formulación se transforma en una optimización sobre un conjunto reducido de parámetros Φ , con la adaptación de $\Delta\theta$ representada como $\Delta\theta(\Phi)$. Esto resulta en la siguiente fórmula ajustada:

$$\max_{\Phi} \sum_{(D, S^*) \in \mathcal{Z}} \sum_{i=1}^{l_{S^*}} \log(p_{\theta_0 + \Delta\theta(\Phi)}(s_i^* | S_{<i}^*, D))$$

Esta reformulación refleja la eficiencia introducida por LoRA en la adaptación de modelos preentrenados, particularmente en tareas de resumen automático, donde el enfoque se centra en optimizar un número mucho menor de parámetros, reduciendo significativamente la carga computacional y los requisitos de memoria.

3.4.3. Aplicación Práctica de LoRA en Modelos de Lenguaje

Una red neuronal contiene múltiples capas densas que realizan multiplicaciones matriciales. Tradicionalmente, estas matrices de peso tienen rango completo. Sin embargo, LoRA propone que durante la adaptación, las actualizaciones a estos pesos también tienen un rango intrínseco "bajo". Para una matriz de peso preentrenada $W_0 \in \mathbb{R}^{d \times k}$, LoRA restringe su actualización representando este cambio con una descomposición de bajo rango $W_0 + \Delta W = W_0 + BA$, donde $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, y el rango $r \ll \min(d, k)$. Durante el entrenamiento, W_0 se mantiene fijo sin recibir actualizaciones de gradiente, mientras que A y B contienen parámetros entrenables. La pasada hacia adelante modificada se expresa como:

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

3.4.3.1. Beneficios y Limitaciones Prácticos

El beneficio más significativo de LoRA es la reducción en el uso de memoria y almacenamiento. Por ejemplo, en un Transformer grande entrenado con Adam, el uso de VRAM se reduce hasta en $2/3$ si $r \ll d_{\text{model}}$, ya que no es necesario almacenar los estados del optimizador para los parámetros congelados.

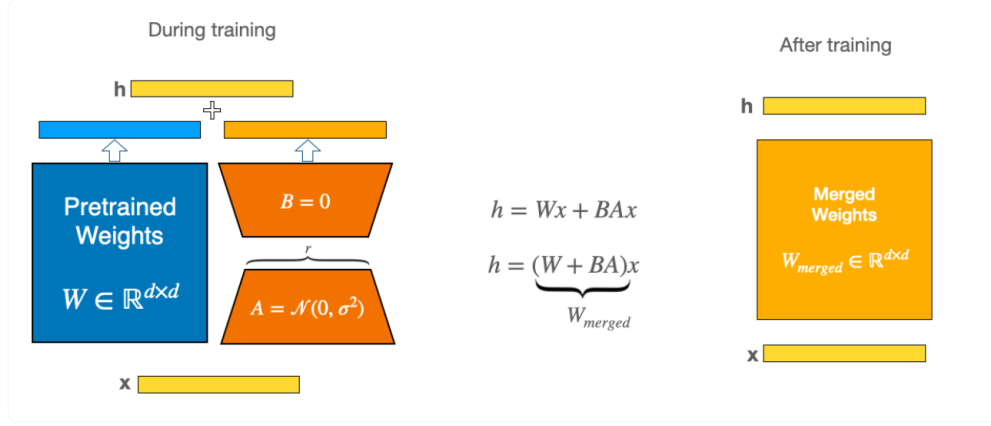


Figura 3.4: Esquema de LoRA durante y post entrenamiento

3.4.4. QLoRA

En la extensión del ajuste fino eficiente de parámetros, QLoRA [6] representa una evolución de LoRA, incorporando cuantificación en bloque y adaptadores de bajo rango para una finura más eficiente en términos de memoria. QLoRA mejora sobre LoRA cuantificando el modelo del transformador a una precisión de 4 bits y utilizando optimizadores paginados para manejar alzas de memoria.

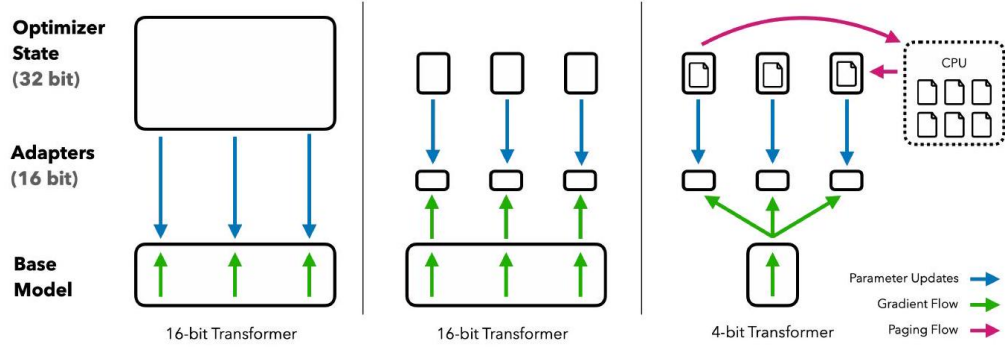


Figura 3.5: Esquema de Ajuste Estandar v/s LoRA v/s QLoRA

3.4.4.1. Cuatificación en Bloque de k-bits

QLoRA implementa una cuantificación en bloque, donde cada bloque del tensor de entrada se cuantifica de manera independiente con su propia constante de cuantificación. Este enfoque aborda el problema de los valores atípicos en la cuantificación, permitiendo una mejor utilización de los bins de cuantificación. La cuantificación y de-cuantificación se formalizan como sigue:

$$\mathbf{X}^{\text{Int8}} = \text{round} \left(\frac{127}{\text{absmax}(\mathbf{X}^{\text{FP32}})} \mathbf{X}^{\text{FP32}} \right)$$

$$\text{dequant} \left(c^{\text{FP32}}, \mathbf{X}^{\text{Int8}} \right) = \frac{\mathbf{X}^{\text{Int8}}}{c^{\text{FP32}}}$$

3.4.4.2. Adaptadores de Bajo Rango

La finura con LoRA se basa en el uso de adaptadores de bajo rango que no requieren actualizar los parámetros preentrenados del modelo completo, reduciendo así los requisitos de memoria. QLoRA extiende este enfoque al integrar adaptadores de bajo rango en un modelo cuantificado, manteniendo la eficiencia sin comprometer la precisión.

3.4.4.3. Requisitos de Memoria y Optimizadores Paginados

QLoRA aborda eficientemente los requisitos de memoria a través de la cuantificación de los constantes de cuantificación y la implementación de optimizadores paginados. Esto permite manejar picos de memoria durante la finura de modelos grandes en una sola máquina, reduciendo el riesgo de errores de memoria.

3.4.4.4. Definición Formal de QLoRA

Definimos QLoRA para una sola capa lineal en el modelo base cuantificado con un solo adaptador LoRA de la siguiente manera:

$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}} \text{doubleDequant} \left(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{NF4}} \right) + \mathbf{X}^{\text{BF16}} \mathbf{L}_1^{\text{BF16}} \mathbf{L}_2^{\text{BF16}},$$

donde $\text{doubleDequant}(\cdot)$ se define como:

$$\text{doubleDequant} \left(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{k-bit}} \right) = \text{dequant} \left(\text{dequant} \left(c_1^{\text{FP32}}, c_2^{\text{k-bit}} \right), \mathbf{W}^{\text{4bit}} \right) = \mathbf{W}^{\text{BF16}},$$

Este enfoque permite una finura de alta fidelidad en 4 bits, manteniendo una eficiencia de memoria notable, y resulta en un modelo que puede cambiar entre tareas con un costo mucho menor.

Código 3.2: Estructura del Modelo PeftModelForCausalLM con QLoRA.

```
1 {
2   "PeftModelForCausalLM": {
3     "base_model": "LoraModel",
4     "components": {
5       "LlamaForCausalLM": {
6         "model": "LlamaModel",
7         "components": {
8           "embed_tokens": "Embedding(32000, 4096)",
9           "layers": {
10            "type": "ModuleList",
11            "details": [
12              "32 x LlamaDecoderLayer",
13              {
14                "self_attn": "LlamaAttention",
15                "components": {
```

```

16         "q_proj": "Linear4bit(in_features=4096, out_features=4096, bias=False)",
17         "k_proj": "Linear4bit(in_features=4096, out_features=4096, bias=False)",
18         "v_proj": "Linear4bit(in_features=4096, out_features=4096, bias=False)",
19         "o_proj": "Linear4bit(in_features=4096, out_features=4096, bias=False)",
20         "rotary_emb": "LlamaRotaryEmbedding()"
21     },
22     "LoRA": {
23         "dropout": "Dropout(p=0.1, inplace=False)",
24         "lora_A": "Linear(in_features=4096, out_features=8, bias=False)",
25         "lora_B": "Linear(in_features=8, out_features=4096, bias=False)"
26     }
27 },
28 {
29     "mlp": "LlamaMLP",
30     "components": {
31         "gate_proj": "Linear4bit(in_features=4096, out_features=11008, bias=False)",
32         "up_proj": "Linear4bit(in_features=4096, out_features=11008, bias=False)",
33         "down_proj": "Linear4bit(in_features=11008, out_features=4096, bias=False)"
34     }
35 },
36 {
37     "input_layernorm": "LlamaRMSNorm()",
38     "post_attention_layernorm": "LlamaRMSNorm()"
39 }
40 ]
41 ]
42 },
43 "norm": "LlamaRMSNorm()"
44 },
45 "lm_head": "CastOutputToFloat(Linear(in_features=4096, out_features=32000, bias=
↳ False))"
46 }
47 }
48 }
49 }

```

3.5. Métricas de Evaluación

Análisis de las métricas de evaluación estándar en resúmenes automáticos, como ROUGE y BERTScore.

La evaluación de los resúmenes automáticos generados por modelos como GPT-4o y LLAMA2-7B-32K-Instruct requiere métricas precisas y robustas que reflejen la calidad del texto generado. Entre estas métricas, BERTScore se ha establecido como un estándar en la evaluación de la similitud semántica entre el resumen de referencia y el generado por el modelo.

3.5.1. ROUGE

La métrica Recall-Oriented Understudy for Gisting Evaluation (ROUGE) fue introducida por primera vez en [7]. ROUGE es esencial para determinar automáticamente la calidad de un resumen comparándolo con resúmenes ideales generados por personas. Esta métrica cuenta las unidades de superposición, como n-gramas, secuencias de palabras y pares de palabras, entre los resúmenes generados por computadora que se evalúan y los resúmenes ideales escritos por humanos.

ROUGE incluye varias medidas:

- **ROUGE-N**: Cuenta el número de unidades de n-grama compartidas por un resumen dado y un grupo de resúmenes de referencia, donde N es la longitud del n-grama. Por ejemplo, ROUGE-1 para unigramas y ROUGE-2 para bigramas.
- **ROUGE-L**: Calcula la estadística de la Secuencia Común más Larga (LCS ó *Longest Common Sequence*) para dos frases dadas, el resumen generado en este caso y el objetivo.

ROUGE es la métrica más utilizada para la evaluación automática de resúmenes. La métrica ROUGE-n se basa en la medida de recuperación y se basa en la comparación de n-gramas. Una serie de n-gramas (principalmente uno y dos) se obtiene de los resúmenes de referencia y del resumen candidato (resumen generado automáticamente). Sea p el número de n-gramas comunes entre el resumen candidato y el resumen de referencia, y q el número de n-gramas extraídos solo del resumen de referencia. La puntuación se calcula como:

$$\text{ROUGE-}n = \frac{p}{q}$$

ROUGE-L emplea el concepto de la secuencia común más larga (LCS) entre dos secuencias de texto. La intuición es que cuanto más larga sea la LCS entre dos frases de resumen, más similares son. Aunque esta métrica es más flexible que la anterior, tiene la limitación de que todos los n-gramas deben ser consecutivos.

3.5.2. BERTScore

BERTScore utiliza representaciones contextuales basadas en embeddings para medir la similitud entre oraciones de referencia y candidatas. La principal ventaja de BERTScore es su capacidad para evaluar la similitud de significado más allá de las coincidencias de palabras exactas.

3.5.2.1. Representación de Tokens y Cálculo de Similitud

Para cada palabra en las oraciones de referencia x y candidatas $\hat{x}$, BERTScore calcula representaciones vectoriales utilizando modelos de embeddings contextuales como BERT o ELMO. Estas representaciones capturan el contexto de cada palabra, lo que permite una evaluación más precisa de la similitud semántica.

La medida de similitud utilizada en BERTScore es la similitud coseno, que se calcula como el producto interno de los vectores normalizados de las palabras. Esta medida considera a las palabras individualmente, pero las representaciones contextuales contienen información del resto de la oración, lo que enriquece la evaluación.

3.5.2.2. Cálculo de BERTScore

BERTScore se calcula a partir de las representaciones vectoriales de las palabras en las oraciones de referencia y candidatas. Para cada palabra en la oración de referencia x , se encuentra la palabra en la oración candidata $\hat{x}$ con la mayor similitud coseno, y viceversa. Estos cálculos de similitud se suman y normalizan para obtener las puntuaciones de precisión, recall y F1, que son:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}.$$

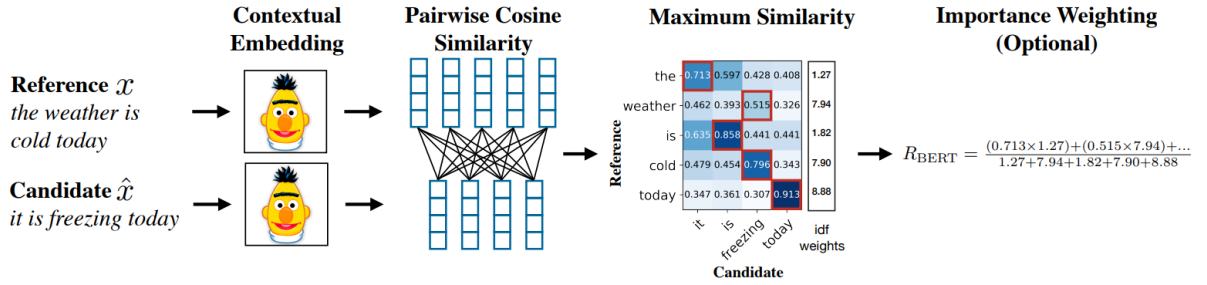


Figura 3.6: Computación de BERTScore [2]

Estas puntuaciones reflejan cuán bien el resumen candidato captura el contenido del resumen de referencia, considerando tanto la cobertura de información (recall) como la precisión de la información incluida (precisión).

Capítulo 4

Resultados

En esta sección, se presentan los resultados obtenidos durante el proceso de ajuste fino de los modelos LLAMA2-7b-32K y GPT-4o en la tarea de resumen de diálogos. Se evaluaron diversos aspectos, incluyendo la calidad de los resúmenes generados, el tiempo de inferencia, el uso de recursos de hardware y el costo de inferencia. Los resultados se comparan y analizan para proporcionar una visión completa de la efectividad y eficiencia de cada modelo.

4.1. Descripción General

El proceso de ajuste fino (fine-tuning) del modelo LLAMA2-7B se llevó a cabo utilizando el conjunto de datos MediaSum. El objetivo fue mejorar la capacidad del modelo para generar resúmenes precisos y coherentes de diálogos extensos. El modelo se entrenó en una instancia de Google Cloud a2-highgpu-4g, que cuenta con 4 GPUs NVIDIA A100. Este entorno proporcionó la capacidad computacional necesaria para manejar el gran volumen de datos y la complejidad del modelo.

4.2. Ingeniería de prompts

La ingeniería de prompts es un aspecto crucial en la generación de resúmenes de diálogos utilizando modelos LLMS como LLAMA2-7B-32K-Instruct y GPT-4o. A continuación, se presentan los prompts utilizados para cada modelo.

4.2.1. LLAMA2-7B-32K-Instruct

El prompt para LLAMA2-7B-32K-Instruct se diseñó para guiar al modelo a extraer información estructurada de diálogos en entrevistas de televisión y radio. El prompt incluye una descripción del sistema, el problema a resolver, y especificaciones claras del formato de salida esperado. Además, se manejan adecuadamente los caracteres especiales para evitar problemas en la extracción de JSON.

System

You are a team of media analysts who need to extract structured information from dialogues in television and radio interviews.

Your task is to generate a structured summary of the dialogues contained in the interview.

You must extract the relevant information from each participant, topics discussed, opinions expressed, and other important elements of the dialogue.

Your response must be a JSON with the structured information from the dialogues contained in the interviews.

Your response is always truthful and accurate.

You will win a huge prize if your response is the most accurate and concise.

Problem Description

- Your task is to produce a succinct and objective summary of the content of the conversation presented in the context.
- Your summary should be limited to describing the main themes and discussion points.
- Your response should be ONLY and ONLY the detailed JSON in the expected output.
- The summary is at most 256 words long.
- Ideally, the summary should be as short as possible, aiming for 64 words or less.
- Your response should be precise and not add information that is not contained in the context.
- NECESSARY: Your response MUST be in a JSON format ended with ""
- IMPORTANT: Apostrophes used in English to indicate possession, such as 'Pablo's work' or similar, should be replaced with another appropriate character that is neither a single quote " ' " nor a double quote " " ", as these cause issues when extracting the JSON. Use ' ' ' instead. For example 'Pablo's work' should be replaced with 'Pablo's work'.

Context

```
"" text
{dialog_history}
""
```

Expected Result

```
"" json
{{
  'summary': '...'
}}
""
```


4.2.2. GPT-4o

El prompt para GPT-4o sigue una estructura similar al de LLAMA2-7B-32K-Instruct, con una descripción del sistema, el problema a resolver, ejemplos de entrada y salida, y especificaciones claras del formato de salida esperado. También se manejan adecuadamente los caracteres especiales para evitar problemas en la extracción de JSON.

```
sys_prompt = """
You are a team of media analysts who need to extract structured information
from dialogues in television and radio interviews.
Your task is to generate
a structured summary of the dialogues contained in the interview.
You must extract the relevant information from each participant, topics
discussed, opinions expressed, and other important elements of the dialogue.
Your response must be a JSON with the structured information from the
dialogues contained in the interviews.
Your response is always truthful and accurate.
You will win a huge prize
if your response is the most accurate and concise.
"""

prompt = f"""
# Problem Description
- Your task is to produce a succinct and objective summary of the content
of the conversation presented in the context.
- Your summary should be limited to describing the main themes and discussion points.
- Your response should be ONLY and ONLY the detailed JSON in the expected output.
- The summary is at most 256 words long.
- Ideally, the summary should be as short as possible, aiming for 64 words or less.
- Your response should be precise and not add information that is not contained
in the context.
- IMPORTANT: Apostrophes used in English to indicate possession, such as
'Pablo's work' or similar, should be replaced with another appropriate
character that is neither a single quote " ' " nor a double quote " " ", as
these cause issues when extracting the JSON. Use ' ' ' instead.

# Examples

## Example 1
- Input
"" text
< context >
""
```

```

- Output
"" json
...
""

## Example 2
- Input
"" text
< context >
""
- Output
"" json
...
""

...

# Context
"" text
{context.strip()}
""

# Expected result
"" json
{{
'summary': < str >
}}
""
"""

```

4.3. Descripción del conjunto de Validación

El conjunto de validación utilizado en este estudio es una muestra del conjunto de datos MediaSum, que contiene 10000 diálogos. Este conjunto de datos es representativo del tipo de diálogos que se espera resumir, ya que incluye diálogos extensos y variados.

4.3.1. Tokens en el conjunto de Validación

A continuación, se presentan los resultados de los tokens en el conjunto de validación para los modelos GPT-4o y LLAMA2-7b-32K-Instruct.

Tabla 4.1: Comparación de tokens en el conjunto de validación para GPT-4o y LLAMA2-7b-32K-Instruct

Tipo de Token	Métrica	GPT-4o	LLAMA2-7b-32K-Instruct
Prompt	Suma	19,980,147	23,461,009
Prompt	Máximo	21,622	25,118
Prompt	Promedio	1,998.0147	2,346.1009
Resumen	Suma	192,850	256,825
Resumen	Máximo	244	282
Resumen	Promedio	19.285	25.6825

4.3.2. Discusión acerca de los tokens

Los resultados muestran que el modelo LLAMA2-7b-32K-Instruct maneja un mayor número de tokens en comparación con GPT-4o. Esto se refleja en todas las métricas: suma, máximo y promedio de tokens tanto para el prompt como para el resumen. La mayor cantidad de tokens manejados por LLAMA2-7b-32K-Instruct podría indicar una mayor capacidad para procesar diálogos extensos y generar resúmenes más detallados. Sin embargo, también podría aumentar el tiempo de inferencia y el uso de recursos de hardware.

Es crucial balancear la calidad del resumen con la eficiencia en términos de tiempo y recursos. Además de la cantidad de tokens, es importante evaluar la calidad de los resúmenes generados, incluyendo métricas como la coherencia, precisión y relevancia del contenido resumido. Finalmente, la discusión debe incluir una evaluación del costo de inferencia, considerando el costo-beneficio al elegir el modelo más adecuado para la tarea específica. ‘

4.4. Resultados del Fine-Tuning

El proceso de fine-tuning se llevó a cabo utilizando el dataset *mediasum_train.json*, que contiene 600 ejemplos. El entrenamiento se realizó durante 3 épocas, con un tamaño de lote instantáneo por dispositivo de 2 y un tamaño de lote total de entrenamiento de 16, utilizando pasos de acumulación de gradiente de 4. El número total de pasos de optimización fue 111, y el número de parámetros entrenables fue 19,988,480.

Tabla 4.2: Resultados de la pérdida y precisión durante el entrenamiento y evaluación

Step	Train Loss	Test Loss
110	0.9177	1.0791

La Figura 4.1 muestra la evolución de la pérdida de entrenamiento a lo largo de los pasos de optimización. Se observa una disminución constante de la pérdida, indicando que el modelo está aprendiendo de manera efectiva.

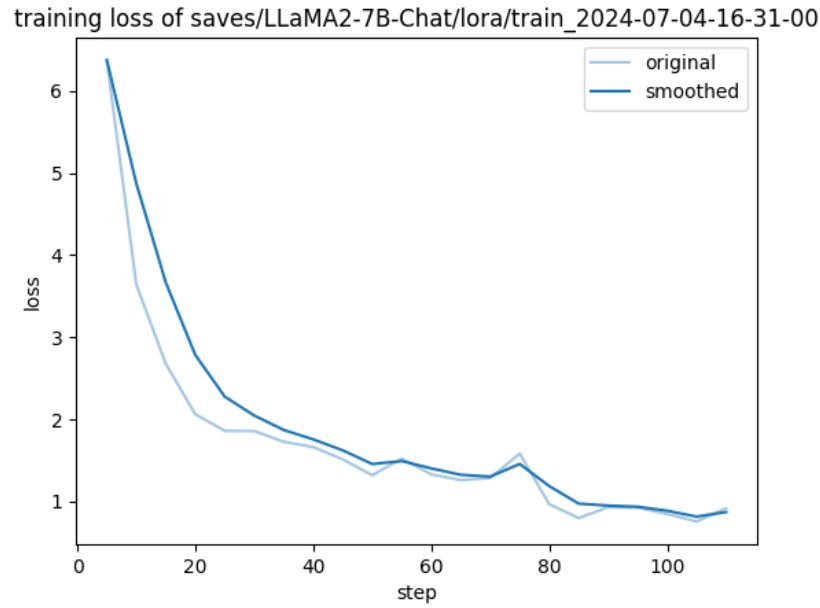


Figura 4.1: Pérdida de entrenamiento del modelo LLAMA2-chat-7b

4.4.1. Eficiencia del Hardware y Throughput de Tokens

El modelo ajustado mostró una eficiencia mejorada en términos de throughput de tokens, alcanzando un throughput de 3796.60 tokens por segundo. Esto sugiere que las optimizaciones realizadas durante el fine-tuning no solo mejoraron el rendimiento del modelo en términos de precisión y pérdida, sino también en términos de eficiencia computacional. La evaluación de muestras por segundo fue de 3.781.

4.5. Evaluación de la Calidad de los Resúmenes

4.5.1. Resultados de BERTScore

La evaluación de la calidad de los resúmenes generados por los modelos LLAMA y GPT se realizó utilizando BERTScore. Los resultados se presentan en la Tabla 4.3.

Tabla 4.3: Resultados de BERTScore para LLAMA y GPT

Modelo	Precisión	Recall	F1
LLAMA	0.838	0.866	0.852
GPT	0.827	0.871	0.848

Los resultados indican que LLAMA tiene una media de precisión de 0.838, un recall de 0.866 y un F1 de 0.852. Por otro lado, GPT muestra una media de precisión de 0.827, un recall de 0.871 y un F1 de 0.848. Aunque LLAMA tiene mejores puntuaciones en precisión y F1, GPT tiene un mejor recall, lo que sugiere que GPT es más efectivo en capturar una mayor cantidad de información

relevante en ciertos contextos específicos.

4.5.2. Resultados de ROUGE

La evaluación de los resúmenes también se realizó utilizando las métricas ROUGE-1, ROUGE-2 y ROUGE-L. Los resultados se presentan en la Tabla 4.4.

Tabla 4.4: Resultados de ROUGE para LLAMA y GPT

Modelo	ROUGE-1	ROUGE-2	ROUGE-L
LLAMA	0.207	0.073	0.168
GPT	0.183	0.061	0.142

Los resultados indican que LLAMA tiene una media de ROUGE-1 de 0.207, ROUGE-2 de 0.073 y ROUGE-L de 0.168. Por otro lado, GPT muestra una media de ROUGE-1 de 0.183, ROUGE-2 de 0.061 y ROUGE-L de 0.142. Estos resultados refuerzan la conclusión de que LLAMA es más efectivo en la generación de resúmenes coherentes y relevantes.

4.6. Evaluación de la Eficiencia de los Modelos

4.6.1. Tiempo de Inferencia

Tabla 4.5: Tiempo de Inferencia para LLAMA y GPT

Modelo	Tiempo de Inferencia
LLAMA	44.08 minutos
GPT	30.95 minutos

El tiempo de inferencia se mide desde el momento en que se envía la solicitud al modelo hasta que se recibe la respuesta completa. Para LLAMA, el tiempo de inferencia fue de 44.08 minutos, mientras que para GPT fue de 30.95 minutos. Esto indica que GPT es más rápido en generar resúmenes en comparación con LLAMA.

4.6.2. Costo de Inferencia

Tabla 4.6: Costo de Inferencia para GPT

Modelo	Costo de Tokens de Entrada	Costo de Tokens de Salida
GPT	\$99.90	\$2.89

Tabla 4.7: Costo de Inferencia para LLAMA

Modelo	Tipo De Instancia	Costo medio por hora (GCP ON-DEMAND)
LLAMA	a2-highgpu-4g	\$15.6725

Para calcular el costo total de inferencia, se multiplicó el tiempo de inferencia por el costo por hora de la instancia de Google Cloud. Para LLAMA, con un tiempo de inferencia de 44.08 minutos, el costo total fue de aproximadamente \$11.51 USD. En el caso de GPT los costos se calculan en función de la suma del costo por procesar los tokens de entrada y salida, lo cual suma \$102.79 USD. Esto representa un ahorro del 88.8 % al utilizar LLAMA en lugar de GPT en este esquema de ajuste fino cuantizado.

4.7. Análisis Comparativo entre Modelos

En el análisis comparativo, es fundamental considerar tanto el rendimiento del hardware como la eficiencia energética de los modelos. LLAMA y GPT se ejecutaron en la misma infraestructura, lo que permite una comparación directa de su eficiencia.

Desde la perspectiva del rendimiento del modelo, LLAMA mostró una mejor precisión y F1 en BERTScore, así como mejores resultados en ROUGE en comparación con GPT. Sin embargo, GPT tuvo un mejor recall en BERTScore, lo que sugiere que es más efectivo en capturar información relevante.

En términos de tiempo de inferencia, GPT fue más rápido que LLAMA, con tiempos de 30.95 minutos y 44.08 minutos, respectivamente. Sin embargo, la diferencia en el tiempo de inferencia debe ser balanceada con la calidad de los resúmenes generados y el costo de inferencia.

El costo de inferencia es un factor crucial en la evaluación comparativa. LLAMA mostró un costo total de \$11.51 USD, mientras que GPT tuvo un costo de \$102.79 USD. Esto representa un ahorro del 88.8 % al utilizar LLAMA en lugar de GPT.

En resumen, aunque GPT es más rápido en términos de tiempo de inferencia, LLAMA ofrece una mejor relación costo-eficiencia y una calidad superior en los resúmenes generados. Esto hace que LLAMA sea una opción más viable para aplicaciones que requieren tanto eficiencia como calidad en la generación de resúmenes.

Capítulo 5

Discusión y Trabajo Futuro

La reducción del "train loss" durante el ajuste fino del modelo es un indicador positivo de que el modelo se está adaptando mejor al formato de salida deseado. Este ajuste fino permite que el modelo capture mejor las características específicas del conjunto de datos MediaSum, lo que se traduce en resúmenes más precisos y coherentes. Sin ajuste fino, los resúmenes generados eran más dispersos y menos enfocados en la información esencial. Con el ajuste fino, los resúmenes se volvieron más concisos y alineados con las expectativas humanas.

La elección de BERTScore como métrica de evaluación se basa en su capacidad para correlacionar mejor con los juicios humanos en comparación con otras métricas tradicionales como BLEU o ROUGE. BERTScore utiliza embeddings contextuales para evaluar la similitud entre las oraciones generadas y las de referencia, lo que permite una evaluación más matizada y precisa de la calidad del texto. Esto es especialmente relevante en tareas de generación de texto, donde la coherencia y la fluidez son tan importantes como la precisión.

El uso de BERTScore se justifica aún más por su robustez frente a ejemplos desafiantes, como se menciona en el trabajo de Zhang et al. (2019) [8]. En el contexto de la generación de resúmenes de diálogos, esta robustez es crucial, ya que los diálogos pueden ser altamente variados y contextualmente ricos. La capacidad de BERTScore para manejar esta variabilidad y aún así proporcionar una evaluación precisa lo convierte en una herramienta invaluable para este estudio.

En términos de trabajo futuro, una dirección prometedora es la exploración de modelos más pequeños pero con ventanas de contexto más grandes, como phi-3. Según Abdin et al. (2023) [9], phi-3-mini, un modelo de 3.8 mil millones de parámetros, ha demostrado un rendimiento comparable a modelos mucho más grandes, lo que sugiere que podría ser una opción viable para aplicaciones con recursos de hardware limitados. Este enfoque podría reducir significativamente los costos de implementación sin sacrificar la calidad del resumen.

Otra área de investigación futura es la evaluación de nuevas métricas de evaluación, como KG-BERTScore y G-EVAL. Wu et al. (2020) [10] proponen KG-BERTScore, que incorpora gráficos de conocimiento para mejorar la evaluación de traducciones automáticas sin referencias. Liu et al. (2023) [11] presentan G-EVAL, que utiliza GPT-4 con un enfoque de "chain-of-thoughts" para evaluar la calidad de textos generados. Estas métricas podrían ofrecer una evaluación aún más precisa y alineada con los juicios humanos, mejorando así la calidad de los resúmenes generados.

Es crucial también considerar el impacto del ajuste fino en la capacidad del modelo para generalizar a nuevos datos. Aunque el ajuste fino mejora la precisión en el conjunto de datos MediaSum, es crucial evaluar cómo se comporta el modelo con otros conjuntos de datos de diálogos. Esto permitirá determinar si las mejoras observadas son específicas del conjunto de datos o si pueden generalizarse a otros contextos. La evaluación en múltiples conjuntos de datos también ayudará a identificar posibles sesgos introducidos durante el ajuste fino.

La elección de BERTScore también se alinea con la necesidad de métricas que puedan capturar la calidad semántica de los resúmenes generados. A diferencia de métricas basadas en coincidencias exactas, BERTScore evalúa la similitud semántica entre las oraciones, lo que es crucial para tareas de resumen donde la reescritura y la parafraseo son comunes. Esta capacidad de capturar la calidad semántica es especialmente útil en el contexto de diálogos, donde la información puede ser presentada de múltiples maneras sin perder su significado esencial.

La robustez de BERTScore frente a ejemplos desafiantes también se extiende a su capacidad para manejar errores tipográficos y variaciones lingüísticas. Esto es particularmente relevante en el contexto de diálogos, donde los errores tipográficos y las variaciones en el lenguaje son comunes. La capacidad de BERTScore para evaluar con precisión la calidad del texto a pesar de estas variaciones lo convierte en una herramienta valiosa para este estudio. Además, su capacidad para correlacionar bien con los juicios humanos lo hace ideal para evaluar la calidad de los resúmenes generados.

En términos de trabajo futuro, otra dirección prometedora es la exploración de diferentes esquemas de aprendizaje. En lugar de utilizar únicamente el aprendizaje supervisado y LORA para entrenar el modelo, se podrían explorar métodos como ReFT (Representation Finetuning). Según Wu et al. (2023)[12], ReFT permite realizar intervenciones específicas en las representaciones ocultas del modelo, lo que puede resultar en una adaptación más eficiente y efectiva del modelo a tareas específicas. Este enfoque podría ofrecer una alternativa más eficiente en términos de parámetros y recursos de hardware.

Finalmente, es importante considerar la evaluación del rendimiento del modelo en diferentes entornos de hardware. Probar el modelo en diferentes GPU y nubes comerciales permitirá evaluar cómo afectan estos factores a los tiempos de inferencia y a la calidad del resumen generado. Además, la evaluación de modelos privados como Gemini y Cloud podría ofrecer perspectivas adicionales sobre el rendimiento y la eficiencia del modelo en diferentes contextos. Esta evaluación ayudará a identificar las configuraciones de hardware más eficientes y rentables para la implementación del modelo.

5.1. Contribuciones

5.1.1. Impacto Económico y Práctico

La implementación de un modelo cuantizado en hardware local emerge como una alternativa económica significativa al uso del API de OpenAI, alineándose con las tendencias contemporáneas de descentralización de recursos computacionales. Este enfoque, centrado en la eficiencia y reducción de costos operativos, promete ser una solución viable para entornos con recursos computacionales limitados o donde se requiere una mayor independencia de servicios externos. El impacto económico de esta implementación se evaluará en función de la disminución de costos operativos y su eficacia en el procesamiento de lenguaje natural.

5.1.2. Avances en el Conocimiento y Habilidades Técnicas

Este proyecto ha contribuido significativamente al conocimiento técnico en el campo de la adaptación y optimización de modelos de lenguaje de gran escala, como LLAMA2. La experiencia acumulada en la adaptación de estos modelos no solo ha mejorado las habilidades técnicas en un área de rápido desarrollo, sino que también ha abierto nuevas posibilidades para futuras investigaciones y aplicaciones prácticas. La comprensión obtenida es invaluable, resaltando la importancia y la versatilidad de los modelos de lenguaje de gran escala en diversas aplicaciones.

Capítulo 6

Conclusión

Este estudio ha comparado la eficiencia y precisión de los modelos LLAMA2-7b-32K-Instruct y GPT-4o en la tarea de resumen de diálogos utilizando el conjunto de datos MediaSum. Los resultados obtenidos muestran que LLAMA2-7b-32K-Instruct tiene una ligera ventaja en precisión y F1 en BERTScore, mientras que GPT-4o tiene un mejor recall. La mayor capacidad de procesamiento de tokens de LLAMA2-7b-32K-Instruct se traduce en resúmenes más detallados y precisos, aunque también implica un mayor uso de recursos de hardware y tiempo de inferencia.

La implementación local de modelos cuantizados en hardware local ha demostrado ser una alternativa económica significativa al uso del API de OpenAI. Este enfoque no solo reduce los costos operativos, sino que también mejora la eficiencia operativa y ofrece una mayor flexibilidad y control sobre el procesamiento de datos. La comparación de costos de inferencia muestra que el uso de LLAMA2-7b-32K-Instruct en un esquema de ajuste fino cuantizado puede resultar en un ahorro del 88.8

El ajuste fino de los modelos ha tenido un impacto significativo en su performance. Este proceso ha permitido que los modelos capturen mejor las características específicas del conjunto de datos MediaSum, resultando en resúmenes más precisos y coherentes. Además, el ajuste fino ha optimizado el uso de recursos de hardware, mejorando la eficiencia computacional y reduciendo los costos operativos.

Este proyecto ha contribuido significativamente al conocimiento técnico en el campo de la adaptación y optimización de modelos de lenguaje de gran escala. La experiencia acumulada ha mejorado las habilidades técnicas en un área de rápido desarrollo y ha abierto nuevas posibilidades para futuras investigaciones y aplicaciones prácticas. La comprensión obtenida es invaluable, resaltando la importancia y la versatilidad de los modelos de lenguaje de gran escala en diversas aplicaciones.

Es crucial reconocer las limitaciones del estudio, como la evaluación en un único conjunto de datos, y la necesidad de evaluar la capacidad del modelo para generalizar a nuevos datos. Las posibles direcciones futuras para la investigación incluyen la exploración de modelos más pequeños con ventanas de contexto más grandes, la evaluación de nuevas métricas de evaluación como KG-BERTScore y G-EVAL, y la exploración de diferentes esquemas de aprendizaje como ReFT.

La robustez de las métricas utilizadas, como BERTScore y ROUGE, ha sido crucial para la evaluación de la calidad de los resúmenes generados. Estas métricas han demostrado ser efectivas

en la evaluación de la calidad del texto, proporcionando una evaluación precisa y alineada con los juicios humanos. La combinación de BERTScore y ROUGE ha permitido una evaluación integral de la calidad de los resúmenes generados.

En resumen, aunque GPT-4o es más rápido en términos de tiempo de inferencia, LLAMA2-7b-32K-Instruct ofrece una mejor relación costo-eficiencia y una calidad superior en los resúmenes generados. Esto hace que LLAMA2-7b-32K-Instruct sea una opción más viable para aplicaciones que requieren tanto eficiencia como calidad en la generación de resúmenes. La implementación local de modelos cuantizados en hardware local es una alternativa económica y eficiente que promete ser una solución viable para entornos con recursos computacionales limitados.

Bibliografia

- [1] Zhu, C., Liu, Y., Mei, J., y Zeng, M., “Mediasum: A large-scale media interview dataset for dialogue summarization,” Journal Name, 2023. Microsoft Cognitive Services Research Group.
- [2] Koh, H. Y., Ju, J., Liu, M., y Pan, S., “An empirical survey on long document summarization: Datasets, models and metrics,” Journal Name, 2023. Monash University, Australia and Deakin University, Australia.
- [3] y Noam Shazeer y Niki Parmar y Jakob Uszkoreit y Llion Jones y Aidan N. Gomez y Lukasz Kaiser y Illia Polosukhin, A. V., “Attention is all you need,” arXiv preprint arXiv:1706.03762, 2017.
- [4] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., y Scialom, T., “Llama 2: Open foundation and fine-tuned chat models,” en Proceedings of the Conference on Open Foundation and Fine-Tuned Chat Models, (San Francisco, CA, USA), pp. 15–17, 2023, doi:<https://doi.org/10.48550/arXiv.2307.09288>.
- [5] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., y y Weizhu Chen, L. W., “Lora: Low-rank adaptation of large language models,” arXiv preprint arXiv:2305.12345, 2023. Microsoft Corporation, {edwardhu, yeshe, phwallis, zeyuana, yuanzhil, swang, luw, wzchen}@microsoft.com, yuanzhil@andrew.cmu.edu.
- [6] Dettmers, T. y y Ari Holtzman y Luke Zettlemoyer, A. P., “Qlora: Efficient finetuning of quantized llms,” arXiv preprint arXiv:1706.12345, 2017. Revised 2023.

- [7] Lin, C.-Y., “Rouge: A package for automatic evaluation of summaries,” en Text Summarization Branches Out, (Barcelona, Spain), pp. 74–81, Association for Computational Linguistics, 2004, <https://www.aclweb.org/anthology/W04-1013/>. Available at Information Sciences Institute, University of Southern California.
- [8] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., y Artzi, Y., “Bertscore: Evaluating text generation with bert,” arXiv preprint arXiv:1904.09675, 2019.
- [9] Abdin, M., Jacobs, S. A., Awan, A. A., *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv preprint arXiv:2305.10498, 2023.
- [10] Wu, Z., Zhang, M., Zhu, M., *et al.*, “Kg-bertscore: Incorporating knowledge graph into bertscore for reference-free machine translation evaluation,” arXiv preprint arXiv:2010.00747, 2020.
- [11] Liu, Y., Iter, D., Xu, Y., *et al.*, “G-eval: Nlg evaluation using gpt-4 with better human alignment,” arXiv preprint arXiv:2306.05685, 2023.
- [12] Wu, Z., Arora, A., Wang, Z., *et al.*, “Reft: Representation finetuning for language models,” arXiv preprint arXiv:2305.10498, 2023.